Secure Password Generator & Analyzer

Advanced Python Command-Line Cybersecurity Project

Student: _____
Course / Institution: _____
Date: _____

Abstract

This project implements a local command-line tool that generates random passwords using Python's cryptographically secure secrets API and analyzes password structure for common weaknesses. It provides an assumption-based entropy estimate, actionable recommendations, and optional zxcvbn guessability feedback. It does not intentionally persist or transmit passwords.

1. Problem Statement

Weak, reused, and predictable passwords can be vulnerable to guessing and credential-stuffing attacks. A local generator and analyzer helps users create unique passwords and understand common weaknesses without submitting passwords to a third-party website.

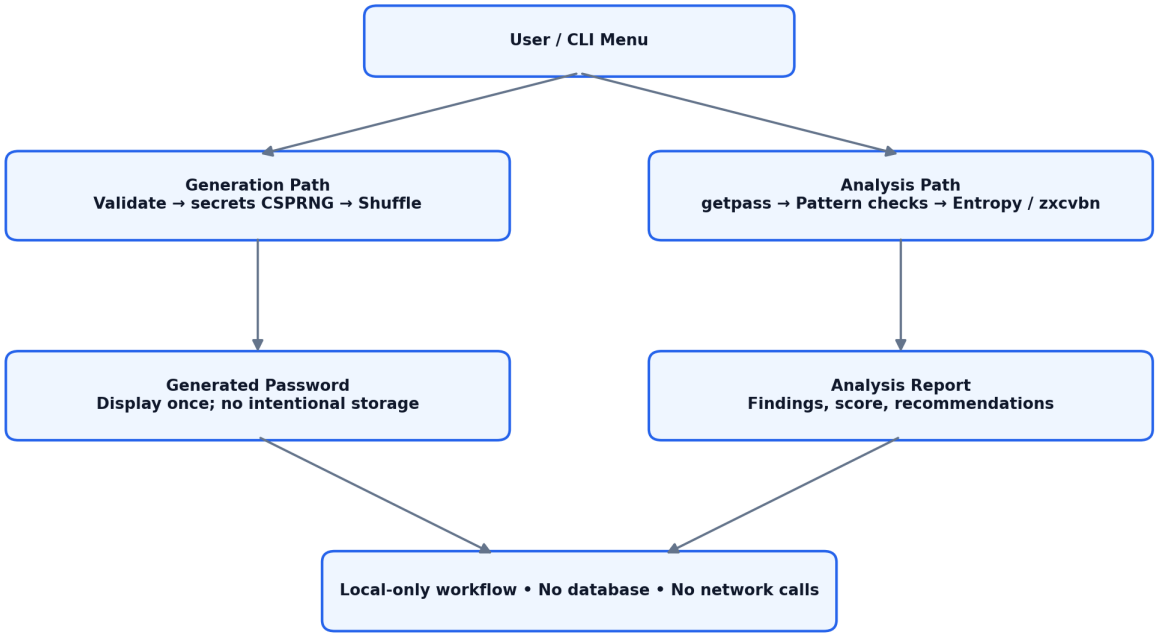
2. Objectives

- Generate passwords using a cryptographically secure random source.
- Guarantee at least one character from each selected character category.
- Detect common examples, repeated characters, and predictable sequences.
- Explain theoretical entropy and its assumptions.
- Provide test coverage and practical security guidance.

3. Scope and Technology

The project is a local educational CLI built with Python 3.10+, using secrets, string, re, math, getpass, and unittest. The optional zxcvbn dependency adds a pattern-aware guessability estimator. It does not authenticate users, manage a credential vault, query breach databases, or store passwords.

4. System Architecture



5. Algorithms

Password generation: Validate the requested length and character sets; select one character from each selected category; fill remaining positions using secrets.choice; shuffle with secrets.SystemRandom; display the generated password.

Entropy approximation: H = L × log■(N), where L is length and N is the inferred character-pool size. This assumes independent, uniformly selected characters from a known pool. Human-selected passwords generally violate this assumption. Because the generator guarantees class inclusion, this calculation is an explanatory approximation, not an exact measurement of its output distribution.

Strength analysis: The custom score combines length thresholds, observed character categories, and pattern warnings. It is a transparent heuristic, not a calibrated security metric. Optional zxcvbn estimates guessability using common patterns but cannot guarantee security.

6. Security Analysis

- Use secrets rather than random for security-sensitive randomness.
- The program does not intentionally log, save, or transmit password input.
- getpass hides interactive input where the terminal supports it.
- Python strings cannot be reliably zeroized from memory.
- Malware, screen capture, terminal recording, clipboard utilities, and a compromised operating system can still expose secrets.
- Crack-time estimates depend on attacker hardware, password-hashing algorithm, and guessing rate.
- Do not use this academic prototype as a password manager; use a reputable password manager for real credentials.

7. Testing

Automated tests cover default length, character-category inclusion, length boundaries, empty character-set rejection, ambiguous-character exclusion, empty input, common-password detection, repeated characters, entropy calculation, and invalid input types. Run: `python -m unittest discover -s tests -v`.

8. Limitations

The custom score and entropy estimate are advisory. zxcvbn is also a model and may not represent every attacker strategy or breach corpus. The application performs no online breach check. A generated password must still be stored safely and never reused. This project has not undergone an independent security audit.

9. Future Enhancements

- Passphrase mode using a vetted word list.
- Privacy-preserving breach screening.
- Accessibility and localization improvements.
- Fuzz testing, threat modeling, and independent security review.

10. Conclusion

The project demonstrates secure random generation, local analysis, input validation, and automated testing. Its most important security feature is the careful use of a cryptographically secure random source and its most important caveat is that no simple score can guarantee password strength. Unique passwords, a reputable password manager, and multi-factor authentication remain essential.